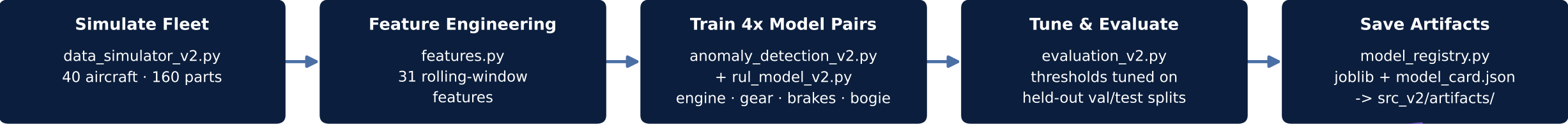
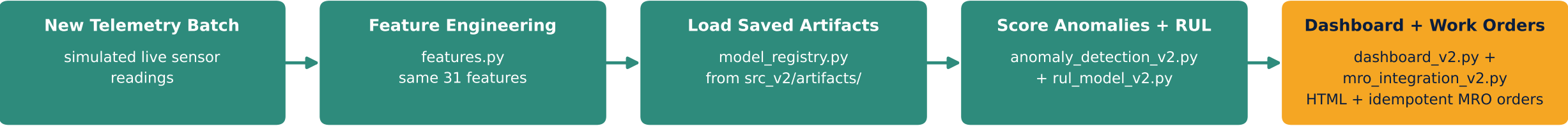


FLEETGUARD AI

System Architecture — Aerospace Predictive Maintenance Pipeline (v2)
TRAIN.PY — offline, occasional (re-run when new data accumulates)



PREDICT_SERVICE.PY — fast, frequent (never retrains)



loads trained
models (no retrain)

Live demo = two console commands:

python train.py then python predict_service.py (from src_v2/) — full runbook on page 2

● Training stage (offline) ● Serving stage (online) ● Action / output (dashboard + MRO)

HACKATHON DEMO RUNBOOK

FleetGuard AI — run this live in under 20 seconds, no internet required after setup

1 Install dependencies (one-time)

```
pip install -r src_v2/requirements_v2.txt
```

numpy, pandas, scikit-learn, joblib — no GPU, no API keys.

2 Move into the source directory

```
cd src_v2
```

3 Train the models (~11s)

```
python train.py
```

Simulates the fleet, engineers features, trains 4x IsolationForest + RandomForestRegressor pairs, tunes thresholds, evaluates, saves artifacts.

4 Serve predictions on new telemetry (~7s)

```
python predict_service.py
```

Loads saved models (no retraining), scores a fresh batch, builds the dashboard, and sends idempotent, retry-safe work orders to the MRO system.

5 Run the unit tests (~2s)

```
python -m unittest tests.test_models -v
```

Confirms all 5 tests pass — feature engineering, anomaly pipeline, and RUL regression.

What to point out live

- RUL error: 4.91 cycles overall (MAE)
- Anomaly recall: 66.7% on held-out test units
- Mean detection lead time: 216 cycles before failure
- 32 auto-generated work orders, 0 duplicates on retry

Where outputs land

- outputs_v2/dashboard_v2.html — fleet health dashboard
- outputs_v2/work_orders_v2.json — MRO work orders
- outputs_v2/model_metrics_v2.json — benchmark report
- src_v2/artifacts/ — trained joblib models + model card